How to Use the KRAKEN API

A practical developer guide to KRAKEN's REST API (Kroger Remote Asset Kit for Enterprise Networking).

> **Scope:** KRAKEN exposes **416 routes across 46 modules — 362 under `/api/`**. This guide covers authentication, conventions, and worked examples for the domains you'll use most. For the complete route list, see `kraken_api_reference.md` (human tables) and `kraken_openapi.json` (Swagger/Postman).

---

1. Base URLs & Environments

Environment	Base URL	Session Cookie
Production	`https://kraken.kroger.com`	`kraken_session`
Dev	`http://<dev-host>:5000`	`kraken_dev_session`

Most data endpoints accept an `env` query parameter to select the dataset:

`env` value	Dataset
`retail` *(default)*	Retail stores (~25 divisions)
`dc`	Non-retail (DC + KF)
`labs`	Lab stores (ci, ht)

```
GET /api/search?mode=standard&q=ci907&env=retail
```

---

2. Authentication (do this first)

Every route except the login/register/logout/static paths sits behind a global `before_request` gate. An unauthenticated call to any `/api/*` path returns:

```
HTTP 401
{ "error": "Not authenticated" }
```

Browser (non-API) routes redirect to `/login` instead.

Logging in

Authentication is **LDAP-backed and session-cookie based** — there are no API keys or bearer tokens. Post your Kroger EUID + password to `/login`, then reuse the session cookie on every subsequent request.

```
import requests

BASE = "https://kraken.kroger.com"
s = requests.Session()

# Authenticate — populates the kraken_session cookie on the session object
r = s.post(f"{BASE}/login", data={"euid": "abcd123", "password": "••••••••"})
r.raise_for_status()

# Every call after this reuses the cookie automatically
me = s.get(f"{BASE}/api/user_perms").json()
print(me)   # {'role': 'operator', 'divs': [...], 'flags': [...], ...}
```

> **Tip:** Hit `GET /api/user_perms` right after login to confirm your role and division scope before doing real work.

Authorization levels

KRAKEN uses a 4-tier role model. The API enforces the *minimum* level per route:

Level	Meaning
login	Any authenticated user
operator	Can perform port actions / device changes
superuser	Elevated management (e.g. tag definitions)
admin	Full control (user mgmt, config deploy, firmware, reload)

If you call an endpoint above your level you'll get `403` with an error message. Notable admin-only domains: `/api/config/*` **(config deploy),** `/api/firmware/*`**,** `/api/admin/users*`**,** `/api/reload`**,** `/api/activity*`**.**

---

3. Request & Response Conventions

- **GET** endpoints take **query-string** parameters. **POST/PUT/PATCH** take a **JSON body** (`Content-Type: application/json`).
- Path parameters use store codes (`ci907`), device names (`ar6405sw1`), or numeric IDs.
- Search-style responses share a common envelope:

```
{
  "results":  [ ["ci907","ar6405sw1","10.x.x.x", "..."], ... ],
  "columns":  ["Store","Device","IP Address","MAC Address","..."],
  "total":    1234,
  "returned": 1000,
  "page":     1,
  "pages":    2,
  "show_all": false,
```

```
  "page_size": 1000
}
```

- `results` is an **array of row arrays**; map them positionally against `columns`.
- Pagination: pass `page=N`, or `all=1` to return everything in one shot (use sparingly on big divisions).
- Some heavy/long-running actions return a **job id** and you poll a status endpoint (see §7).
- Streaming endpoints (`/api/fresh`, `/api/ping_stream`) use **Server-Sent Events** (`text/event-stream`).

---

4. The Search API (the workhorse)

A single endpoint, `GET /api/search`, dispatches to **13 search modes** plus a natural-language `ai` mode.

```
GET /api/search?mode=<mode>&q=<query>&div=<div>&env=<env>&page=1&all=0
```

`mode`	Purpose	Uses `q2`?
`standard`	Device name / free-text / "type in store"	–
`comment`	TUNA comment search (device + comment)	✔ `q2` = comment terms
`whitelist`	DHCP whitelist (macaddr) devices	–
`mac`	MAC lookup (supports `^` / `$` anchors)	–
`ip`	IP substring or `re:` regex mode	–
`vlan`	VLAN id or name	–
`switchport`	Search by SwitchPort field	–
`stale`	Devices not pinged in N days	✔ `q2` = days
`devcount`	Count devices per store matching a term	–
`compare`	Stores that have **both** A and B	✔ `q2` = term B
`missing`	Stores that have A but **not** B	✔ `q2` = term B
`diff`	Device diff between two stores	✔ `q2` = store 2
`porthistory`	Port change history	–
`ai`	Natural language → auto-picks mode	–

Examples

```
# All switches in one store
s.get(f"{BASE}/api/search",
      params={"mode":"standard","q":"switches in ci907","env":"retail"})

# MACs ending in BCCA across a division
s.get(f"{BASE}/api/search", params={"mode":"mac","q":"BCCA$","div":"ci"})
```

```
# Devices stale > 30 days
s.get(f"{BASE}/api/search", params={"mode":"stale","q":"","q2":"30","div":"ci"})

# IP range via regex
s.get(f"{BASE}/api/search",
      params={"mode":"ip","q":"re:10\\.148\\.1\\.2[1-5]$","div":"ci"})

# Natural language
s.get(f"{BASE}/api/search",
      params={"mode":"ai","q":"printers in cincinnati not pinged in a month"})
```

The `ai` mode also returns an `ai_intent` object showing how it parsed your query — handy for debugging.

Scope filter

Any search accepts a `scope` parameter to restrict to a set of stores (comma/space separated). Validate a raw scope string first with:

```
POST /api/scope/parse   { "raw": "ci907, ci910, 01400915" }
→ { "valid":[...], "invalid":[...], "total":n }
```

---

5. Common Read Workflows

Store health & inventory

```
GET /api/store_health?store=ci907&env=retail        # summary counts
GET /api/store_overview/ci907                        # dashboard overview
GET /api/store_switches/ci907                        # switch list
GET /api/store_macs/ci907                            # MAC → device map
GET /api/store/ci907/ip-segments                     # VLAN-grouped IP view
```

MAC lookup across everything

```
GET  /api/lookup/mac/AABBCCDDEEFF                    # single, all environments
POST /api/lookup/mac_batch   { "macs": ["AABB...","CCDD..."] }
```

Division rollups

```
GET /api/divisions_summary
GET /api/division_overview/ci
GET /api/division_devices/ci/<category>
```

Switch detail (POST with a body)

```
s.post(f"{BASE}/api/device/portmap",
       json={"store":"ci907","device":"ar6405sw1","ip":"10.x.x.x"})

s.post(f"{BASE}/api/device/environment",
       json={"store":"ci907","device":"ar6405sw1","ip":"10.x.x.x"})
```

Port Inspector (deep single-port view)

```
GET /api/portmap/ci907/ar6405sw1/port/1%2F3%2F13
```

(URL-encode slashes in the port id as `%2F`.)

---

6. Write / Action Endpoints

These change device state — most require **operator+**. Always confirm scope with `/api/user_perms` first.

SwitchPort actions

```
s.post(f"{BASE}/api/switch/toggle_port",

json={"store":"ci907","device":"ar6405sw1","ip":"10.x","port":"1/3/13","action":"disable"})

s.post(f"{BASE}/api/switch/bounce_poe",  json={...})
s.post(f"{BASE}/api/switch/batch",       json={...})   # multi-port, one session per switch
```

ECS port compliance

```
GET  /api/ecs/ci907                # list violations
POST /api/ecs/ci907/enable         # { ports:[...] }
POST /api/ecs/ci907/disable
POST /api/ecs/ci907/gdctest        # operator+ VLAN3030 route test (background job)
```

Ping

```
# Batch (frontend chunks to 25; server-wide semaphore caps concurrency)
s.post(f"{BASE}/api/ping_batch",
       json=[{"ip":"10.x.x.1","rowIndex":0}, {"ip":"10.x.x.2","rowIndex":1}])

s.get(f"{BASE}/api/ping_once", params={"ip":"10.x.x.1"})   # one-shot
```

Notes, tags, issues

```
POST   /api/notes                { store, device, mac, note }
PUT    /api/store/ci907/tags     { tags:[...] }          # operator+
```

```
POST   /api/issues                { store, device, ... }  # create issue
PATCH  /api/issues/<id>           { status:"resolved" }
```

7. Long-Running Jobs (background pattern)

Heavy operations (config deploy, firmware, ECS scans, cable diag, GDC route tests) run **asynchronously** to avoid the IIS ARR 30-second timeout. The pattern is always the same:

1. **Kick off** → returns `{ "job_id": "..." }`
2. **Poll status** until `done` / `failed`
3. **Fetch/export** results

```
# 1. Start a bulk cable diagnostic
job = s.post(f"{BASE}/api/device/bulk_cable_diag", json={...}).json()
jid = job["job_id"]

# 2. Poll
import time
while True:
    st = s.get(f"{BASE}/api/device/bulk_cable_diag/job/{jid}").json()
    if st["status"] in ("completed","failed"): break
    time.sleep(3)

# 3. Download the XLSX
xlsx = s.get(f"{BASE}/api/device/bulk_cable_diag/download/{jid}")
open("diag.xlsx","wb").write(xlsx.content)
```

Other job families follow the same shape:

Domain	Start	Poll
Config deploy	`POST /api/config/deploy`	`GET /api/config/jobs/<job_id>`
Firmware	`POST /api/firmware/{scan,stage,reboot}`	`GET /api/firmware/job/<job_id>`
ECS scan	`POST /api/ecs/bulk_scan`	`GET /api/ecs/job/<job_id>`
ECS GDC test	`POST /api/ecs/ci907/gdctest`	`GET /api/ecs/gdc_job/<job_id>`

8. Streaming (Server-Sent Events)

`/api/fresh` and `/api/ping_stream` stream `text/event-stream`. Consume line-by-line:

```
with s.get(f"{BASE}/api/fresh", params={"store":"ci907","env":"retail"}, stream=True) as r:
    for line in r.iter_lines():
        if line and line.startswith(b"data: "):
            import json
            evt = json.loads(line[6:])
            print(evt["status"], evt.get("message",""))
            if evt["status"] in ("done","error"): break
```

9. Exporting Data

```
# Server-side XLSX build from any result set
resp = s.post(f"{BASE}/api/export",
              json={"columns": cols, "rows": rows, "mode": "search"})
open("kraken_export.xlsx","wb").write(resp.content)
```

Several domains have their own exports too: `GET /api/issues/export`, `GET /api/tags/export`, `GET /api/activity/export` (admin).

10. Admin-Only Endpoints

Require **admin** role (and `/admin` also gates behind a password form for the UI):

```
GET    /api/admin/users                # list users + parsed perms
POST   /api/admin/users                # add/update  { euid, name, role, divs, flags, email }
DELETE /api/admin/users/<euid>         # remove (can't delete self)
POST   /api/reload                     # reload TUNA data into memory
GET    /api/activity                   # audit log (filters: euid, action, severity,
date_from/to)
GET    /api/server_status              # full status JSON
```

Config-deploy and firmware management are entirely admin-gated — see `/api/config/*` and `/api/firmware/*` in the full reference.

11. Error Handling Cheatsheet

Status	Meaning	Fix
`401 Not authenticated`	No/expired session	Re-`POST /login`
`403`	Role too low, or admin password not entered	Check `/api/user_perms`; use an authorized account
`400`	Missing/invalid params	Verify required fields & store format (`^[a-z]{2}\d{3}$`)
`404`	Unknown resource id	Check the id / store code
`500`	Backend/SNMP/SSH failure	Retry; inspect message; device may be unreachable

Store codes are validated as **two letters + three digits** (e.g. `ci907`). Numeric `DIV00STR` codes (e.g. `01400907`) are auto-normalized to letter codes on search endpoints.

12. Quickstart (copy/paste)

```
import requests, time

BASE = "https://kraken.kroger.com"
s = requests.Session()
s.post(f"{BASE}/login", data={"euid":"YOUR_EUID","password":"YOUR_PW"}).raise_for_status()

# Who am I?
print(s.get(f"{BASE}/api/user_perms").json())

# Find all switches in a store
r = s.get(f"{BASE}/api/search",
          params={"mode":"standard","q":"switches in ci907","env":"retail"}).json()
cols = r["columns"]
for row in r["results"]:
    print(dict(zip(cols, row)))

# Pull live port data for the first switch
first = dict(zip(cols, r["results"][0]))
pm = s.post(f"{BASE}/api/device/portmap",
            json={"store":"ci907","device":first["Device"],"ip":first["IP Address"]}).json()
print(f"{len(pm.get('ports',[]))} ports")
```

Appendix — Where to look things up

Need	File
Human-readable route tables by module	`kraken_api_reference.md`
Machine spec (import into Swagger UI / Postman)	`kraken_openapi.json`
Regenerate after code changes	`py kraken_api_catalog.py C:\TUNAFIN_WEB --base-url https://kraken.kroger.com`

Load `kraken_openapi.json` into Postman or Swagger UI for a clickable, testable catalog of every endpoint with its auth level and path parameters.